

Agent运行模式

Hermes Agent 运行模式

一、概述

Hermes Agent 是一个功能完备的多模态 AI Agent 系统，支持多种运行模式以适配不同场景（本地 CLI 交互、远程 Gateway 服务、Honcho 持久化记忆集成等）。与常见的"Agent 模式 vs Plan 模式"二分法不同，Hermes 将**模式**概念分散在多个独立维度中，包括：API 调用模式、记忆写入模式、记忆召回模式、写入频率模式、CLI 输入模式、会话策略模式、推理级别等。这些维度可以自由组合，形成一个多维的配置空间。

核心文件列表：

文件	作用
run_agent.py	AIAgent 主类，所有模式的核心执行逻辑
honcho_integration/client.py	Honcho 客户端配置，记忆/召回模式定义
honcho_integration/cli.py	Honcho CLI 命令处理
honcho_integration/session.py	Honcho Session 管理器
hermes_cli/main.py	CLI 主入口，honcho 子命令注册
gateway/run.py	Gateway 服务， /plan 命令入口
agent/skill_commands.py	/plan 等 slash command 构建逻辑
toolsets.py	工具集（Toolset）系统定义
hermes_state.py	SQLite 会话持久化存储
cli.py	旧版 CLI 入口，display/tool_progress 模式
gateway/config.py	Gateway 平台配置

二、API 调用模式（api_mode）

2.1 模式定义

所在位置： run_agent.py:587-609 ， AIAgent.__init__ 中的 api_mode 属性

API 调用模式决定了 Hermes 如何与底层 LLM 提供商通信。三种互斥模式：

模式	值	说明
OpenAI 兼容对话补全	chat_completions	标准 OpenAI /v1/chat/completions API，默认值
OpenAI Codex Responses	codex_responses	OpenAI Responses API（用于 GPT 系列模型，支持更丰富的 tool calling）
Anthropic 原生消息	anthropic_messages	Anthropic /v1/messages API（用于 Claude 原生调用）

2.2 自动检测逻辑

```
# run_agent.py AIAgent.__init__ 源码逻辑
if api_mode in {"chat_completions", "codex_responses", "anthropic_messages"}:
    self.api_mode = api_mode          # 显式传入
elif self.provider == "openai-codex":
    self.api_mode = "codex_responses" # OpenAI Codex 提供商
elif "chatgpt.com/backend-api/codex" in self._base_url_lower:
    self.api_mode = "codex_responses" # Codex URL
```

```

elif self.provider == "anthropic" or "api.anthropic.com" in self._base_url_lower:
    self.api_mode = "anthropic_messages" # Anthropic 直接调用
elif self._base_url_lower.rstrip("/").endswith("/anthropic"):
    self.api_mode = "anthropic_messages" # 第三方兼容端点 (MiniMax、DashScope 等)
else:
    self.api_mode = "chat_completions" # 默认: OpenAI 兼容

# Direct OpenAI URL → 升级到 Responses API (GPT-5.x tool calling 拒绝 chat/completions)
if self.api_mode == "chat_completions" and self._is_direct_openai_url():
    self.api_mode = "codex_responses"

```

2.3 模式对核心行为的影响

2.3.1 Prompt Caching 启用条件

```

# run_agent.py:667-669
is_openrouter = self._is_openrouter_url()
is_claude = "claude" in self.model.lower()
is_native_anthropic = self.api_mode == "anthropic_messages"
self._use_prompt_caching = (is_openrouter and is_claude) or is_native_anthropic

```

- anthropic_messages 模式：始终启用 prompt caching
- OpenRouter + Claude 模型：启用 prompt caching（通过 OpenRouter 的缓存博元）
- chat_completions / codex_responses：不启用缓存

2.3.2 API 请求构建差异

```

# run_agent.py:4617-4660
if self.api_mode == "anthropic_messages":
    # 使用 Anthropic SDK 的 messages.create()
    anthropic_messages = self._prepare_anthropic_messages_for_api(api_messages)
    return build_anthropic_kwargs(model=..., messages=anthropic_messages, ...)

if self.api_mode == "codex_responses":
    # 使用 OpenAI Responses API (tools 参数结构不同)
    payload_messages = api_messages # 无 system wrapper
    instructions = ""

if self.api_mode == "chat_completions":
    # 标准 OpenAI chat completions (messages[0] 注入 system prompt)
    ...

```

2.3.3 流式响应处理差异

```

# run_agent.py:3912-3930
if self.api_mode == "codex_responses":
    # Codex 流由 _run_codex_stream 内部处理
    result["response"] = self._run_codex_stream(api_kwargs, ...)

if self.api_mode == "anthropic_messages":
    # Anthropic SDK 原生流: client.messages.stream()
    ...

```

2.3.4 凭证刷新机制差异

API 模式	凭证刷新方法
anthropic_messages	_try_refresh_anthropic_client_credentials()
chat_completions + Nous	_try_refresh_nous_client_credentials()
codex_responses + OpenAI Codex	_try_refresh_codex_client_credentials()

2.3.5 Fallback 激活时的模式切换

```
# run_agent.py:4388-4402
fb_api_mode = "chat_completions"
if fb_provider == "openai-codex":
    fb_api_mode = "codex_responses"
elif fb_provider == "anthropic" or fb_base_url.endswith("/anthropic"):
    fb_api_mode = "anthropic_messages"
elif self._is_direct_openai_url(fb_base_url):
    fb_api_mode = "codex_responses"

self.api_mode = fb_api_mode # 同步切换 API 模式
```

✍ Note

Fallback 切换 = 联合约束重绑定（Joint Constraint Rebinding）。

一次更新 model、provider、base_url、api_mode 四个属性，同时 context_compressor 也会按 fallback 模型的 context_length 重新按比例计算 threshold_tokens。

问题：如果只更新 model 而忘记更新 context_compressor，primary 的 200K 阈值会被错误复用给小容量的 fallback 模型（32K），导致会话直接 overflow。

关键：相关状态必须一起更新，否则约束系统会不一致。

✍ Note

Prompt Caching 的启用条件 = 平台特性嗅探（Platform Feature Sniffing）。

实际是三元联合条件：

- anthropic_messages 模式：平台保证支持，始终启用
- OpenRouter + Claude：需双重交叉验证（提供商名 + 模型名）

原因：OpenRouter 聚合了多种后端，缓存博元是否可用取决于实际路由到的后端而非平台本身。若只检查 is_openrouter，会错误地为非 Claude 模型也启用缓存。

三、记忆模式（Memory Mode）

3.1 模式定义

配置位置： honcho_integration/client.py:81-136 ， HonchoClientConfig 类

记忆模式决定了 Hermes 如何持久化 Agent 的跨会话记忆，与 Honcho 云端记忆服务的交互策略相关。

模式	值	行为
混合模式	hybrid （默认）	同时写入 Honcho 云端和本地 MEMORY.md / USER.md
仅 Honcho	honcho	仅写入 Honcho 云端，禁用本地 markdown 文件写入

模式	值	行为
仅本地	local	仅使用本地 markdown 文件（Honcho 相关写入跳过）

 Note

三种记忆模式对应三种经典的一致性模型：

- **hybrid = 写后复制（Write-Ahead + Write-Through）**：同时写两个副本，兼顾云端可迁移性和本地可读性。一方失败时需 try/except 补偿。
- **honcho = 远程持久化优先**：本地仅作只读缓存，类 CDN 源站优先策略，适合多设备同步。
- **local = 本地写入即提交**：零网络依赖，无法跨设备共享，单节点数据库模型。

honcho 模式通过软禁用而非硬删除代码路径实现，是**条件跳过（Conditional Skip）**策略。

3.2 配置层级与优先级

Honcho 配置支持多层级覆盖，优先级从高到低：

```
hosts[hermes].memoryMode  （主机级，显式配置）
    ↓ （若不存在）
global memoryMode          （根级）
    ↓ （若不存在）
"hybrid"                    （硬编码默认值）
```

同时支持按 **Peer** 覆盖：

```
# honcho_integration/client.py:70-78
def _resolve_memory_mode(global_val: str | dict, host_val: str | dict | None) -> dict:
    val = host_val if host_val is not None else global_val
    if isinstance(val, dict):
        default = val.get("default", "hybrid")
        overrides = {k: v for k, v in val.items() if k != "default"}
    else:
        default = str(val) if val else "hybrid"
        overrides = {}
    return {"memory_mode": default, "peer_memory_modes": overrides}

# honcho_integration/client.py:104-109
def peer_memory_mode(self, peer_name: str) -> str:
    return self.peer_memory_modes.get(peer_name, self.memory_mode)
```

配置示例（JSON 格式，honcho.json）：

```
{
  "memoryMode": {
    "default": "hybrid",
    "hermes": "honcho",
    "other-peer": "hybrid"
  }
}
```

3.3 运行时行为差异

```
# run_agent.py:1119-1131
if self._honcho_config and self._honcho:
    _hcfg = self._honcho_config
    _agent_mode = _hcfg.peer_memory_mode(_hcfg.ai_peer)
    _user_mode = _hcfg.peer_memory_mode(_hcfg.peer_name or "user")

    if _agent_mode == "honcho":
        self._memory_flush_min_turns = 0      # 不强制等待，直接禁用
        self._memory_enabled = False          # 本地 MEMORY.md 写入完全禁用
        logger.debug("peer %s memory_mode=honcho: local MEMORY.md writes disabled", _hcfg.ai_peer)

    if _user_mode == "honcho":
        self._user_profile_enabled = False     # USER.md 写入禁用
        logger.debug("peer %s memory_mode=honcho: local USER.md writes disabled", _hcfg.peer_name
or "user")
```

 Note

两条正交写入管道，不对称的记忆可见性。

MEMORY.md 和 USER.md 是两条完全独立的状态机：各自有自己的 enabled 标志和 flush 守卫，互不干扰。

效果：peer A（AI侧）设为 honcho 只禁用 Agent 记忆写入，peer B（用户侧）的 USER.md 管道完全不受影响。实现上无需 "if ai vs user" 的分支判断，只依赖两个预先绑定好的独立标志位。

本质：关注点分离（Separation of Concerns）在数据流层面的体现。

3.4 CLI 管理接口

```
# 查看当前记忆模式
hermes honcho mode

# 设置记忆模式
hermes honcho mode hybrid    # 写入 Honcho + 本地文件
hermes honcho mode honcho    # 仅 Honcho（跳过本地文件）

# 交互式设置（向导）
hermes honcho setup
# 记忆模式选项：
#   hybrid - write to both Honcho and local MEMORY.md (default)
#   honcho - Honcho only, skip MEMORY.md writes
```

四、记忆召回模式（Recall Mode）

4.1 模式定义

配置位置： honcho_integration/client.py:47-54， run_agent.py:2394-2400

Recall 模式控制Honcho 记忆如何在推理时被使用，即"模型能看到什么记忆信息"。

模式	值	行为
混合召回	hybrid（默认）	自动注入上下文 + Honcho 工具可用（模型自行决定）
仅上下文	context	仅自动注入上下文，Honcho 工具从工具列表中移除
仅工具	tools	不自动注入上下文，但 Honcho 工具可用

Note

召回模式 = "谁来承担检索计算量"的分配问题。

计算位置：

- context → Agent 启动时（预取）→ 延迟 = Prefetch RTT → Token 开销 = 固定
- tools → 模型推理过程中（按需）→ 延迟 = 模型决定 → Token 开销 = 可变
- hybrid → 两者兼有

context = **贪婪预取 (Greedy Prefetch)**：假设记忆一定有用，提前加载。优点：推理时零额外延迟。缺点：预取内容不相关则浪费 token。

tools = **懒加载 (Lazy Loading)**：模型按需判断何时需要记忆。精确度高，但有调用延迟。

hybrid = 预取降低首轮延迟 + 保留按需工具调用的灵活性。

4.2 运行时行为

```
# run_agent.py:2394-2400
if hcfig.recall_mode == "context":
    self._strip_honcho_tools_from_surface() # 移除 honcho_* 工具
    print(" Honcho active - recall_mode: context (Honcho tools hidden)")
else:
    print(f" Honcho active - recall_mode: {hcfig.recall_mode}")

# run_agent.py:6135-6139
_recall_mode = (self._honcho_config.recall_mode if self._honcho_config else "hybrid")
if self._honcho and self._honcho_session_key and _recall_mode != "tools":
    prefetched_context = self._honcho_prefetch(original_user_message)
    if prefetched_context:
        self._honcho_context = prefetched_context # 注入到系统提示词
```

4.3 系统提示词注入差异

```
# run_agent.py:2613-2631
recall_mode = hcfig.recall_mode if hcfig else "hybrid"
honcho_block = (
    "# Honcho memory integration\n"
    f"Active. Session: {self._honcho_session_key}. "
    f"Mode: {mode}. Write frequency: {freq}. Recall: {recall_mode}.\n"
)

if recall_mode == "context":
    honcho_block += (
        "Honcho context is injected into this system prompt below. "
        "All memory retrieval comes from this context - no Honcho tools "
        "are available. Answer questions about the user, prior sessions, "
        "and recent work directly from the Honcho Memory section.\n"
    )
elif recall_mode == "tools":
    honcho_block += "Honcho tools:\n honcho_context <question> - ask Honcho a question, LLM-
synthesized answer\n"
    " honcho_search - semantic search (no LLM)\n"
```

```
" honcho_profile           - peer card snapshot (no LLM)\n"
" honcho_conclude          - write conclusion to memory (no LLM)\n"
```

4.4 Honcho 工具集

当 recall_mode != "context" 时，以下工具对模型可用：

工具名	功能	LLM 调用
honcho_context	向 Honcho 提问，获得 LLM 综合回答	是
honcho_search	对历史记录做语义搜索	否
honcho_profile	获取用户 peer 卡片快照	否
honcho_conclude	将结论/事实写入 Honcho 记忆	否

📌 Note

`_strip_honcho_tools_from_surface()` = 初始化阶段的 Capability Stripping。

在初始化时将 honcho_* 工具从工具列表中删除，而非在执行时检查权限。模型在工具列表中根本看不到这些工具 → 不会尝试调用 → 不会产生调用失败和重试开销。

比运行时守卫（guard）更彻底：守卫是"看到了但被拒绝"，Capability Stripping 是"从不存在"。

效果：工具选择困惑（tool selection confusion）降为零。

五、写入频率模式（Write Frequency）

5.1 模式定义

配置位置： honcho_integration/client.py:110-112

写入频率控制 Honcho 记忆的写入时机和方式。

模式	值	行为
异步写入	async （默认）	后台线程异步写入，不阻塞响应，不消耗 token
同步写入	turn	每轮对话后同步写入，阻塞直到完成
会话结束时批量写入	session	仅在会话结束时批量 flush
每 N 轮写入	整数 N	每 N 轮对话后写入一次

📌 Note

写入频率 = 批量处理策略，核心权衡：延迟 vs 吞吐量 vs 丢失风险。

模式	写入次数/会话	丢失风险	适用场景
turn	N	极低	关键信息不可丢失
async	~N	低（可重试）	日常会话
session	1	高（崩溃即丢失）	短会话/只读
N（整数）	~会话长度/N	中	自定义批量大小

async 模式 = 最终一致性 + Write-Behind Caching：写入不立即对后续查询可见，有限时间内保证成功。后台线程是 SPSC 队列的入队端，下一轮对话触发 flush → 典型的异步流水线。

PS：

- 1. 对应规则在 session 启动时确定 - 从配置文件读取 write_frequency，存入 self._write_frequency
- 2. 整个 session 周期内保持不变 - _write_frequency 没有 setter，运行时无法动态修改
- 3. 非自适应 - 代码中没有任何基于系统负载、网络延迟、token 消耗等指标的动态调整逻辑
- 4. 仅 async 模式有重试机制 - 后台线程失败后会 sleep 2秒重试一次，第二次失败则丢弃

5.2 运行时实现

```
# run_agent.py:5036-5041
_hcfg = getattr(self, '_honcho_config', None)
if _hcfg and _hcfg.peer_memory_mode(_hcfg.ai_peer) == "honcho":
    return # honcho-only 模式跳过本地 flush

effective_min = min_turns if min_turns is not None else self._memory_flush_min_turns
if self._user_turn_count < effective_min:
    return # 轮次不足，跳过 flush

# honcho_integration/session.py
# HonchoSessionManager 根据 write_frequency 决定：
#   async → 提交到后台线程
#   turn   → 同步 write
#   session → 仅在 shutdown 时批量写入
```

5.3 CLI 管理接口

```
# 交互式设置 (hermes honcho setup)
hermes honcho setup
# 写入频率选项：
#   async   - background thread, no token cost (recommended)
#   turn    - sync write after every turn
#   session - batch write at session end only
#   N       - write every N turns (e.g. 5)
```

六、推理级别（Dialectic Reasoning Level）

6.1 模式定义

配置位置： honcho_integration/client.py:116-118

Dialectic Reasoning 控制 Honcho AI-to-AI 推理的深度，用于会话开头的上下文综合。

级别	值	行为描述
最低	minimal	最少推理，几乎透传
低	low（默认）	轻度推理，适合大多数场景
中	medium	中等深度推理
高	high	高强度推理
最高	max	最大推理深度

✍️ Note

Dialectic reasoning = "为理解而计算（Compute-for-Understanding）"，是一个 anytime 算法。

`minimal` → `max` 五个级别决定了 Honcho dialectic endpoint 做多少次 AI-to-AI 推理来综合用户画像。更高推理深度 = 更多 token 消耗，但产出的上下文摘要质量更好。

这是典型的 **anytime algorithm**：低级别立即返回近似结果，高级别在时间预算内持续改进。`prefetch_dialectic` 是后台线程（非阻塞），结果在下一轮对话才被消费——**异步流水线**。

`dialectic_max_chars` 的截断逻辑值得注意：`result[:max_chars].rsplit(" ", 1)[0] + " ..."` —— 在字符边界向前找最近空格截断，保证截断后不残留残词。是**语义保真的有界截断**而非硬切。

6.2 CLI 管理接口

```
# 查看/设置推理级别
hermes honcho peer                # 显示当前设置
hermes honcho peer --reasoning medium # 设置为 medium

# 同时设置 Dialectic cap（注入系统提示词的最大字符数）
hermes honcho tokens --dialectic 800 # 设置 dialectic 结果最大 800 字符
```

Note

Dialectic 是一个跨会话的对等体建模（Peer Modeling）系统。

它不是检索记忆，而是 Honcho 让 AI 侧 peer 和 User 侧 peer 做一轮 AI-to-AI 对话，综合出“用户是谁、用户在意什么、用户工作模式”的高层摘要。

这在 ML 中类似**用户 embedding 的动态精化**：每次会话结束时，用对话历史精化对用户的表示，注入下一会话的系统提示词。区别在于这里是对话驱动（dialogue-driven）的推理，而非统计 embedding。

七、会话策略（Session Strategy）

7.1 模式定义

配置位置： `honcho_integration/client.py:127-128`

会话策略决定如何将工作目录映射到 Honcho 会话。

策略	值	行为
按目录	<code>per-directory</code> （默认）	每个工作目录一个会话
按会话	<code>per-session</code>	每次运行新建会话，按 Hermes session ID 命名
按仓库	<code>per-repo</code>	每个 Git 仓库（用仓库根目录名）一个会话
全局	<code>global</code>	所有目录共享单一会话

Note

`per-repo` 用 Git 仓库根目录作为隔离边界，是**低成本高收益的启发式策略**——天然规避跨仓库污染，无需手动配置白名单。

7.2 CLI 管理接口

```
# 会话映射管理
hermes honcho sessions      # 列出已知目录→会话映射
hermes honcho map <name>    # 将当前目录映射到指定会话名
```

 Note

Session 名称是工作目录的确定性函数，天然支持会话重建。

`per-directory` → 目录名；`per-repo` → Git 仓库名；`per-session` → Hermes `session_id`。这意味着相同的物理环境总是映射到同一个 **Honcho session**，无需维护显式映射表。

手动映射（`hermes honcho map <name>`）是运行时覆盖，优先级高于自动解析——允许同一目录临时重定向到不同的会话上下文。这是一种**临时绑定覆盖（Temporary Binding Override）**，不改自动策略，只改当次语义。

八、CLI 显示模式与交互模式

8.1 busy_input_mode

配置位置： `cli.py:1068-1069`，`run_agent.py` 通过 `self.busy_input_mode` 使用控制"当 Agent 正在输出时，用户按 Enter 键的行为"：

模式	值	行为
中断模式	<code>interrupt</code> （默认）	Enter 键发送中断信号，中断 Agent 当前运行
队列模式	<code>queue</code>	Enter 键将输入排入队列，Agent 完成后处理

```
# cli.py:1068-1069
_bim = CLI_CONFIG["display"].get("busy_input_mode", "interrupt")
self.busy_input_mode = "queue" if str(_bim).strip().lower() == "queue" else "interrupt"
```

 Note

busy_input_mode：队列模式 = 有界缓冲区，中断模式 = 丢弃策略。

- `queue` 模式：Enter 键输入进入 FIFO 队列，Agent 完成后依次处理 → **有界队列（bounded queue）**，队列满时的策略未定义（取决于实现，可能阻塞或丢弃）。
- `interrupt` 模式：Enter 键直接发送中断信号，丢弃当前运行中的任何中间状态 → **中断丢弃（interrupt-and-drop）**，激进但简单。

这实际上是**输入处理策略的选择**：队列模式适合长任务（用户多次输入不丢失），中断模式适合短任务（快速中断重新开始）。

8.2 tool_progress_mode

配置位置： `cli.py:1057-1060`

控制工具调用执行过程中的进度显示模式：

模式	值	行为
<code>off</code>	<code>off / False</code>	关闭工具进度显示
<code>new</code>	<code>new</code>	仅显示新工具调用
<code>all</code>	<code>all</code> （默认）	显示所有工具调用
<code>verbose</code>	<code>verbose</code>	详细模式（包含日志前缀）

YAML 1.1 将裸词 `off` 解析为布尔值 `False`，代码中有兼容处理：

```
_raw_tp = CLI_CONFIG["display"].get("tool_progress", "all")
self.tool_progress_mode = "off" if _raw_tp is False else str(_raw_tp)
```

8.3 resume_display

配置位置： cli.py:1061–1062

控制恢复历史会话时的显示模式：

模式	值	行为
full	full（默认）	展示完整历史消息
minimal	minimal	仅展示单行摘要

8.4 streaming 模式

配置位置： cli.py:1073–1074

控制是否将 LLM 返回的 token 流式输出到终端：

```
self.streaming_enabled = CLI_CONFIG["display"].get("streaming", False)
```

 Note

`streaming_enabled` 的本质是令牌级流式交付 vs 批量组装。

关闭 streaming → LLM 的 token 序列在服务端完整组装后才一次性返回（批处理），用户看到完整回答；开启 streaming → 每个 token 实时 push 到客户端（流水线），用户体验到“逐字输出”。

这涉及**背压（backpressure）**问题：当模型生成速度快于网络带宽时，流式管道需要缓冲或阻塞。streaming 模式下每次 `on_first_delta` 回调的延迟即是背压的度量。

PS：实际场景中，当模型流式输出过慢或者响应不稳定时，可能由于内容不完整导致tool/function call解析失败，这种情况下可以用非流式输出暂时规避该问题

九、Toolset（工具集）系统

9.1 工具集定义

配置位置： toolsets.py:72–300

Toolset 是工具的逻辑分组，Agent 初始化时可指定启用或禁用哪些工具集：

```
# CLI 启动时指定工具集
hermes --toolsets web,terminal
python cli.py --toolsets web,terminal,vision
```

9.2 内置工具集

工具集名	包含工具	说明
web	web_search, web_extract	Web 搜索与内容提取
search	web_search	仅 Web 搜索
vision	vision_analyze	图像分析
image_gen	image_generate	图片生成

工具集名	包含工具	说明
terminal	terminal , process	终端和进程管理
file	read_file , write_file , patch , search_files	文件操作
browser	browser_navigate , browser_snapshot , ...	浏览器自动化
memory	todo , memory	任务管理与记忆
skills	skills_list , skill_view , skill_manage	Skill 管理
tts	text_to_speech	语音合成
code	execute_code , delegate_task	代码执行与子代理委托
mcp	MCP 协议相关工具	MCP 工具生态
honcho	honcho_context , honcho_profile , honcho_search , honcho_conclude	Honcho 记忆工具
research	组合工具集	综合研究
development	组合工具集	开发相关
full_stack	组合工具集	全栈工具

PS：当工具过多时，可以参考CC的tool查找工具，进行懒加载

9.3 复合工具集

工具集可以包含其他工具集：

```
TOOLSETS = {
    "research": {
        "description": "Comprehensive research toolkit",
        "tools": [],
        "includes": ["web", "file", "memory"]
    },
}
```

9.4 Toolset 过滤逻辑

```
# run_agent.py:926-947
self.tools = get_tool_definitions(
    enabled_toolsets=enabled_toolsets,    # 仅有这些工具集
    disabled_toolsets=disabled_toolsets,  # 排除这些工具集
    quiet_mode=self.quiet_mode,
)

if enabled_toolsets:
    print(f"    ✅ Enabled toolsets: {' , '.join(enabled_toolsets)}")
if disabled_toolsets:
    print(f"    ❌ Disabled toolsets: {' , '.join(disabled_toolsets)}")
```

🔗 Note

Toolset include = DAG 组合 + 拓扑排序，循环检测在初始化阶段而非运行时。

research includes ["web", "file", "memory"] → 系统解析依赖图并展开为最终工具列表。按依赖顺序展开，检测 A includes B, B includes A 循环，对重复出现的工具去重。

在初始化阶段检测冲突 → Agent 执行到一半才发现工具不可用的问题是**不可能发生的**。

enabled + disabled = 白名单优先，黑名单兜底：白名单 n ~黑名单。

白名单存在时只启用白名单中的工具集，黑名单在其基础上排除指定项。这与 Linux 包管理器的 `apt install X + apt exclude Y` 机制相同。

好处：在预定义工具集上做细粒度调整，无需重新定义整个集合。

十、Plan 模式（/plan 命令）

10.1 架构概述

重要澄清： `/plan` 严格意义上不是一个独立的 Agent 运行模式（如“Plan Mode: 开启”），而是一个 **slash command**（斜杠命令），通过加载 `/plan skill` 来执行专门的规划工作流。

核心理解：

- `/plan` 本质上是一个 **skill 加载机制**，而非独立的 Agent 模式
- 触发时，它将 `plan skill` 的内容打包成一条系统消息，注入到 Agent 的推理流程中
- 技能内容 + `runtime_note` 共同引导模型执行规划工作流并输出到指定文件

10.2 当前触发方式（手动）

CLI 处理（cli.py:4030–4054）：

```
def _handle_plan_command(self, cmd: str):
    """Handle /plan [request] – load the bundled plan skill."""
    parts = cmd.strip().split(maxsplit=1)
    user_instruction = parts[1].strip() if len(parts) > 1 else ""

    plan_path = build_plan_path(user_instruction)  # 生成默认计划文件路径
    msg = build_skill_invocation_message(
        "/plan",
        user_instruction,
        task_id=self.session_id,
        runtime_note=(
            "Save the markdown plan with write_file to this exact relative path "
            f"inside the active workspace/backend cwd: {plan_path}"
        ),
    )
    # 将消息放入输入队列，等待 Agent 推理处理
    self._pending_input.put(msg)
```

Gateway 处理（gateway/run.py:1900–1920）：

```
if canonical == "plan":
    from agent.skill_commands import build_plan_path, build_skill_invocation_message

    user_instruction = event.get_command_args().strip()
    plan_path = build_plan_path(user_instruction)  # 生成默认计划文件路径
    event.text = build_skill_invocation_message(
        "/plan",
        user_instruction,
        task_id=_quick_key,
        runtime_note=(
```

```

        "Save the markdown plan with write_file to this exact relative path "
        f"inside the active workspace/backend cwd: {plan_path}"
    ),
)
canonical = None # 清空, 跳过普通命令处理

```

两端的共同模式:

1. 解析用户指令
2. 调用 `build_plan_path()` 生成计划文件路径
3. 调用 `build_skill_invocation_message()` 加载 skill 并构建激活消息
4. 将消息注入 Agent 的推理管道

10.3 计划文件路径生成规则

```

# agent/skill_commands.py:21-39
def build_plan_path(user_instruction: str = "", *, now: datetime | None = None) -> Path:
    slug_source = (user_instruction or "").strip().splitlines()[0] if user_instruction else ""
    slug = _PLAN_SLUG_RE.sub("-", slug_source.lower()).strip("-")
    if slug:
        slug = "-".join(part for part in slug.split("-")[:8] if part)[:48].strip("-")
    slug = slug or "conversation-plan"
    timestamp = (now or datetime.now()).strftime("%Y-%m-%d_%H%M%S")
    return Path(".hermes") / "plans" / f"{timestamp}-{slug}.md"

```

生成格式: `.hermes/plans/<年-月-日_时分秒>-<slug>.md`

10.4 Skill 加载机制

```

# agent/skill_commands.py:220-255
def build_skill_invocation_message(cmd_key: str, ...):
    commands = get_skill_commands() # 扫描 ~/.hermes/skills/
    skill_info = commands.get(cmd_key) # 查找 /plan skill
    loaded = _load_skill_payload(skill_info["skill_dir"], task_id=task_id)
    # 构建激活消息: 包含 [SYSTEM: user invoked "plan" skill] + skill 内容

```

10.5 实现自动触发的可能性

虽然 `/plan` 当前是手动触发的 slash command, 但底层机制使其具备自动触发的能力:

方式一: Hook 机制自动触发

项目支持 hooks 配置 (`.claude/settings.json`), 可以设置自动化规则:

```

{
  "hooks": {
    "on_user_message": [
      "when: pattern matches 'implement|build|create|添加功能|实现'",
      "then: auto_invoke /plan"
    ]
  }
}

```

方式二: 基于规则自动触发

在 cli.py 中添加检测逻辑，当用户请求涉及特定模式时自动调用：

```
# cli.py 命令处理逻辑中的自动触发点
def _auto_detect_plan_trigger(self, user_input: str) -> bool:
    """检测是否应自动触发 plan 模式"""
    trigger_patterns = [
        r"implement",
        r"build.*feature",
        r"添加.*功能",
        r"实现.*需求",
        # 更多模式...
    ]
    return any(re.search(p, user_input.lower()) for p in trigger_patterns)
```

方式三：修改 System Prompt 让模型自行决策

在系统提示词中添加指令，让模型在检测到需要规划的场景时主动建议使用 /plan：

```
# config.yaml
system_prompt_overrides:
  - when: "task complexity > threshold"
    inject: "Hint: This task may benefit from planning. Consider suggesting /plan."
```

Note

/plan 的触发机制 = 懒加载的命令模式（Lazy Command Loading）。

/plan 与其他 slash command 的本质区别在于：它不执行特定操作（如 /help 打印帮助），而是**加载一个完整的 skill workflows 注入到推理管道中。**

这意味着 /plan 的"执行"实际上是模型自行完成的——skill 内容引导模型按照规划流程思考，runtime_note 告知模型输出位置。命令处理器只是准备了输入数据。

从这个角度看，**自动化触发 /plan 只需改变"何时准备输入数据"的时机**，而非改变执行逻辑本身。

十一、子代理委托模式（Delegation）

11.1 委托参数

配置位置： run_agent.py:5319–5328，tools/delegate_tool.py

通过 delegate_task 工具创建子代理，支持以下参数：

参数	类型	说明
goal	str	子代理目标描述
context	str	额外上下文
toolsets	list[str]	子代理可用的工具集（默认 [terminal, file, web]）
tasks	list	子任务列表
max_iterations	int	子代理最大工具调用轮次（默认 45）

11.2 委托配置

```
# config.yaml
delegation:
  max_iterations: 45      # 子代理最大轮次
  default_toolsets:      # 子代理默认工具集
    - terminal
    - file
    - web
  model: ""              # 子代理模型（空=继承父模型）
  provider: ""           # 子代理提供商（空=继承）
  base_url: ""           # 直接 OpenAI 兼容端点
  api_key: ""            # 端点 API key
```

 Note

子代理 = Fork-Join 并行模型，max_iterations = 深度限制防止无限递归。

delegate_task 后：父进程（等待子代理结果）和子进程（独立执行）分叉 → 子完成 → 父收集结果继续。标准的 Fork-Join。

max_iterations: 45 = **深度优先搜索的深度限制**，防止复杂任务中无限递归。

两个 Agent 共享父进程的上下文注入（context 参数），但各自有独立工具集和模型配置 → **受限的共享内存模型**：避免全局状态竞争，同时允许子任务有独立能力边界。

 Note

子代理默认工具集 [terminal, file, web] = 最小权限原则（Principle of Least Privilege）。

父 Agent 默认全部工具集，子代理只能操作终端、文件、网络。无法调用 delegate_task（防嵌套委托）或 honcho_*（防记忆交叉污染）。

更多能力 → 必须显式传入 toolsets 参数 → 强制开发者**有意识地选择**子代理能力边界，而非默认全开。

十二、平台模式（Platform Modes）

12.1 支持的平台

配置位置： run_agent.py:546–547 ， AIAgent.platform 属性

平台	值	说明
CLI	cli （默认）	命令行交互界面
Telegram	telegram	Telegram 消息平台
Discord	discord	Discord 平台
WhatsApp	whatsapp	WhatsApp 平台
Feishu	feishu	飞书平台
Mattermost	mattermost	Mattermost 平台
其他	各种字符串	扩展平台

12.2 平台感知行为

平台信息注入到系统提示词，影响格式化提示：


```
# run_agent.py:546-547
self.platform = platform # "cli", "telegram", "discord", "whatsapp", etc.
# 用于注入平台特定的格式化提示
```

Note

平台只是一个字符串标识符，通过系统提示词而非代码路径来适配行为。

新增一个平台（无需修改 Agent 核心逻辑）：只需注入一段格式化的系统提示词，告诉模型“你在 Telegram 上，消息有长度限制，回复不应过长”之类。平台标识符是**开放集（Open Set）**——代码不枚举所有平台，任意字符串都接受。

这是一种**数据驱动的行为适配（Data-Driven Behavior Adaptation）**：行为差异来自提示词内容，而非代码分支。若平台差异需要超越提示词的更深层改变（如消息长度限制的强制校验），才需要真正的代码分支——目前代码中暂无此需求。

十三、其他配置维度

13.1 推理配置（Reasoning Config）

```
# run_agent.py:541-542
reasoning_config: Dict[str, Any] = None # 覆写推理配置
# 例如: {"enabled": True, "effort": "medium"}
# OpenRouter: effort=none 禁用 thinking
# 默认: {"enabled": True, "effort": "medium"}
```

13.2 压缩模式（Compression Mode）

```
# config.yaml
compression:
  enabled: True # 启用上下文压缩
  threshold: 0.50 # 在 50% 上下文限制时触发
  summary_model: "" # 摘要模型（空=使用主模型）
```

Note

threshold: 0.50 = 水印触发（Watermark Trigger），而非被动响应。

在上下文达到 50% 时就开始准备压缩，而非等到 90%+。原因：压缩操作本身涉及摘要模型调用，有额外延迟——必须提前触发才能在上下文耗尽前完成，用户无感知。

本质：**主动水位管理**，防止瓶颈操作拖垮整个流水线。

额外设计：`summary_model` 可独立配置 → 压缩流程是一个**双模型管道**：主模型处理当前任务，摘要模型异步处理历史上下文。

13.3 智能模型路由（Smart Model Routing）

```
# config.yaml
smart_model_routing:
  enabled: False
  max_simple_chars: 160 # 简单查询字符上限
```

```
max_simple_words: 28          # 简单查询词数上限
cheap_model: {}               # 便宜模型配置
```

Note

智能模型路由 = 成本感知的启发式调度器（Cost-Aware Heuristic Scheduler）。

`max_simple_chars: 160` 和 `max_simple_words: 28` 是简单查询的启发式定义：两个维度都低于阈值 → 路由到便宜模型。

劣势：长度不等于复杂度，这个任务难度的判断方式还是挺粗糙的

13.4 Approval 模式

```
# config.yaml
approval:
  auto_approved: []          # 自动批准的工具白名单
  auto_denied: []           # 自动拒绝的工具黑名单
  max_tool_calls: 50         # 每轮最大 RPC 调用数
```

Note

`max_tool_calls: 50` = 防止无限循环的安全阀（Safety Valve），对应图的最长路径限制。

执行图：用户消息 → Agent 推理 → 工具调用 → 工具结果 → Agent 推理 → ... → 可能无限延伸。`max_tool_calls` 将图的深度硬截断 → 强制 50 步内给出最终回答。

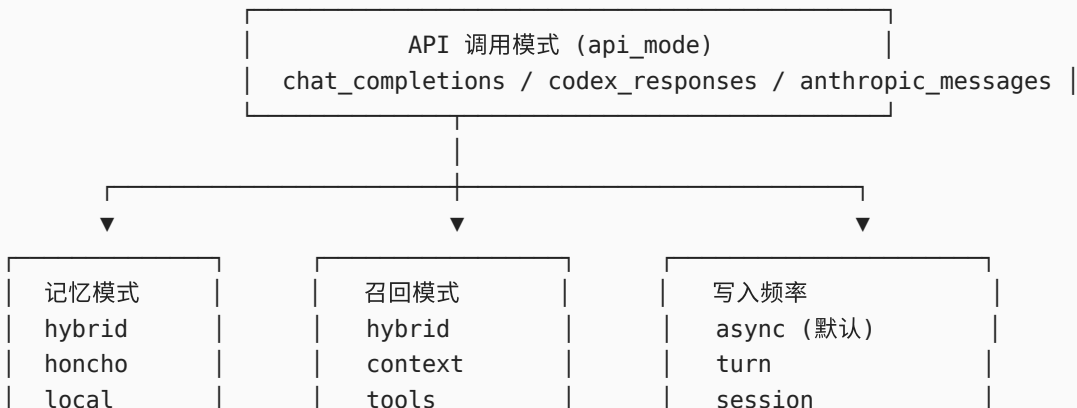
类比：迭代加深（Iterative Deepening）——在深度限制内搜索，找不到解则报告失败而非继续深入。

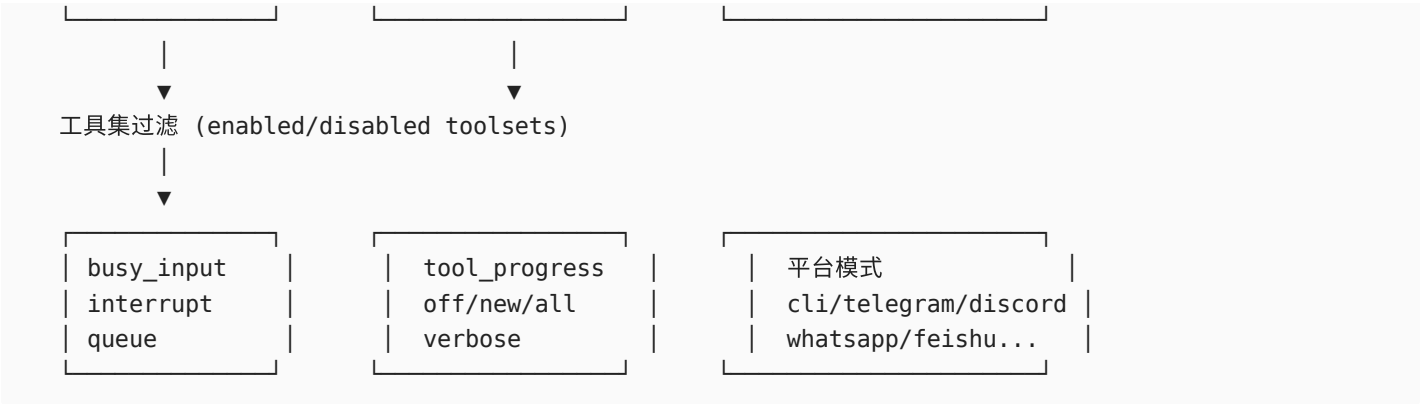
加上 `auto_approved` / `auto_denied` 白名单/黑名单，构成分层访问控制矩阵：高风险工具（文件写入、进程执行）需人工审批；低风险工具（搜索、只读）自动放行。

13.5 Honcho 诊断工具模式

```
# 诊断输出包含所有运行模式配置
hermes doctor
# 输出示例：
# workspace=xxx mode=hybrid freq=async
```

十四、模式交互矩阵





十五、关键代码索引

模式维度	核心定义位置	运行时生效位置
API 调用模式	run_agent.py:587–609	run_agent.py:4617 (_build_api_kwargs)
记忆模式	honcho_integration/client.py:99	run_agent.py:1119–1131
召回模式	honcho_integration/client.py:125	run_agent.py:2394, 6135
写入频率	honcho_integration/client.py:112	honcho_integration/session.py
推理级别	honcho_integration/client.py:118	honcho_integration/session.py
会话策略	honcho_integration/client.py:127	honcho_integration/client.py:resolve_session_na
busy_input_mode	cli.py:1068–1069	cli.py REPL 事件循环
tool_progress_mode	cli.py:1057–1060	cli.py 显示逻辑
Plan 模式	gateway/run.py:1900–1920	agent/skill_commands.py
Toolset 过滤	run_agent.py:654–655	run_agent.py:926–947

十六、总结

Hermes Agent 的"运行模式"并非单一概念，而是一个**多维正交配置空间**：

1. **API 层**：通过 `api_mode` 选择对话协议（OpenAI Chat、OpenAI Responses、Anthropic Messages）
2. **记忆层**：`memory_mode` × `recall_mode` × `write_frequency` 三个维度控制 Honcho 云端记忆的读写策略
3. **工具层**：Toolset 过滤决定 Agent 能调用哪些工具
4. **交互层**：`busy_input_mode`、`tool_progress_mode`、`streaming` 等控制人机交互体验
5. **工作流层**：`/plan` 等 slash command 触发专门的 skill 工作流
6. **平台层**：平台标识符影响系统提示词格式和消息处理方式

这些维度可以任意组合，使得 Hermes 能够从 CLI 交互（`chat_completions` + `hybrid` 记忆 + `async` 写入）到 Telegram Bot（`chat_completions` + `context` 召回 + Telegram 平台）再到高度隔离的开发环境（`anthropic_messages` + `honcho` 记忆 + `local` 会话策略）等各种场景。

Note

整体来看，Hermes Agent 的模式设计体现了三个核心架构哲学：

1. **关注点分离（Separation of Concerns）**：

每个维度控制一个正交的关注面——API 协议、记忆存储、记忆检索、工具能力、人机交互、平台适配——彼此独立演化。新增一个平台（如新 IM 工具）只需注入平台标识符，不改核心逻辑。

2. 延迟绑定 (Lazy Binding):

很多决策延迟到运行时确定——API 模式通过 URL 指纹自动检测、召回模式通过条件跳过实现软删除、Toolset 通过 include DAG 动态展平。减少启动时硬编码依赖，提升可配置性和可扩展性。

3. 分层安全模型 (Layered Security Model):

approval.auto_denied 黑名单、Toolset 能力过滤、max_tool_calls 深度限制、子代理最小权限默认工具集——安全控制分布在多个层级。即使某一层被绕过，其他层仍能提供保护。

算法视角：自适应工作流引擎。 系统在各维度动态感知环境（URL 指纹、模型名、平台类型），在多个约束（token 预算、延迟、安全）下调度（智能路由、压缩触发、工具过滤），在多个目标（低延迟、低成本、高可靠性）之间寻求平衡。